2. Implementation of Classical Encryption Techniques: Develop programs for Caesar Cipher and substitution techniques to understand basic encryption concepts.

To implement classical encryption techniques such as Caesar Cipher and Substitution Cipher using Python and understand the basic concepts of encryption and decryption.

Objectives

1. To understand the concept of cryptography.
2. To learn encryption and decryption processes.
3. To implement Caesar Cipher algorithm.
4. To implement Substitution Cipher algorithm.
5. To understand how plaintext is converted into ciphertext.
6. To analyze the strengths and limitations of classical encryption techniques.

Learning Outcomes

After completing this practical, students will be able to:

- Understand classical cryptographic techniques.
- Implement Caesar Cipher and Substitution Cipher.
- Perform encryption and decryption operations.
- Analyze simple cryptographic algorithms.
- Understand the importance of secure communication.

What is Cryptography?

Cryptography is the science of securing information by converting readable data into an unreadable format.

Basic Terms

Term	Meaning
Plaintext	Original readable message
Ciphertext	Encrypted unreadable message
Encryption	Converting plaintext into ciphertext
Decryption	Converting ciphertext back to plaintext
Key	Secret value used for encryption and decryption

Technique 1: Caesar Cipher

Introduction

The Caesar Cipher is one of the oldest encryption techniques.

It was developed by the Julius Caesar for secure military communication.

Each letter is shifted by a fixed number of positions in the alphabet.

Example

Plaintext:

HELLO

Key:

3

Encryption Process:

H → K
E → H
L → O
L → O
O → R

Ciphertext:

KHOOR

Algorithm

Encryption

1. Read plaintext.
2. Read shift key.
3. Shift each character by key positions.
4. Generate ciphertext.

Decryption

1. Read ciphertext.
2. Read shift key.
3. Shift characters backward.
4. Recover plaintext.

Program 1: Caesar Cipher

```
# ================================================
# CAESAR CIPHER IMPLEMENTATION
# ================================================
```

```
# Function for Encryption
def encrypt(text, shift):

    result = ""

    for char in text:

        # Encrypt uppercase letters
        if char.isupper():

            result += chr((ord(char) + shift - 65) % 26 + 65)

        # Encrypt lowercase letters
        elif char.islower():

            result += chr((ord(char) + shift - 97) % 26 + 97)

        else:
            result += char

    return result


# Function for Decryption
def decrypt(text, shift):

    result = ""

    for char in text:

        if char.isupper():

            result += chr((ord(char) - shift - 65) % 26 + 65)

        elif char.islower():

            result += chr((ord(char) - shift - 97) % 26 + 97)

        else:
            result += char

    return result


# User Input

plaintext = input("Enter Message: ")

shift = int(input("Enter Key Value: "))
```

```
ciphertext = encrypt(plaintext, shift)

print("\nEncrypted Message:", ciphertext)

decrypted = decrypt(ciphertext, shift)

print("Decrypted Message:", decrypted)
```

Sample Output

```
Enter Message: HELLO
Enter Key Value: 3

Encrypted Message: KHOOR
Decrypted Message: HELLO
```

Detailed Code Explanation

ord()

```
ord('A')
```

Output:

```
65
```

Returns ASCII value.

chr()

```
chr(65)
```

Output:

```
A
```

Converts ASCII value back to character.

Formula Used

Encryption:

$C=(P+K)\bmod 26$

Where:

- C = Ciphertext
- P = Plaintext
- K = Key

Decryption:

$P=(C-K) \bmod 26$

```python
# ================================================
# CAESAR CIPHER IMPLEMENTATION
# ================================================

# Function for Encryption
def encrypt(text, shift):

    result = ""

    for char in text:

        # Encrypt uppercase letters
        if char.isupper():

            result += chr((ord(char) + shift - 65) % 26 + 65)

        # Encrypt lowercase letters
        elif char.islower():

            result += chr((ord(char) + shift - 97) % 26 + 97)

        else:
            result += char

    return result


# Function for Decryption
def decrypt(text, shift):

    result = ""

    for char in text:
```

```python
# Function for Decryption
def decrypt(text, shift):

    result = ""

    for char in text:

        if char.isupper():

            result += chr((ord(char) - shift - 65) % 26 + 65)

        elif char.islower():

            result += chr((ord(char) - shift - 97) % 26 + 97)

        else:
            result += char

    return result


# User Input

plaintext = input("Enter Message: ")

shift = int(input("Enter Key Value: "))

ciphertext = encrypt(plaintext, shift)

print("\nEncrypted Message:", ciphertext)

decrypted = decrypt(ciphertext, shift)
```

```
Enter Message:  HELLO
Enter Key Value: 3

Encrypted Message:  KHOOR
Decrypted Message:  HELLO
```

Technique 2: Substitution Cipher

Introduction

In a Substitution Cipher, every plaintext character is replaced by another character according to a predefined mapping.

Example Mapping

Plain	Cipher
A	Q
B	W
C	E
D	R
E	T

Example

Plaintext:

HELLO

Ciphertext:

ITSSG

Program 2: Substitution Cipher

```
# ================================================
# SUBSTITUTION CIPHER IMPLEMENTATION
# ================================================

alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

substitution = "QWERTYUIOPASDFGHJKLZXCVBNM"

encrypt_dict = {}
```

```
decrypt_dict = {}

# Create Encryption Dictionary

for i in range(len(alphabet)):
    encrypt_dict[alphabet[i]] = substitution[i]

# Create Decryption Dictionary

for i in range(len(alphabet)):
    decrypt_dict[substitution[i]] = alphabet[i]

# Input

plaintext = input("Enter Message: ").upper()

# Encryption

ciphertext = ""

for char in plaintext:

    if char in encrypt_dict:
        ciphertext += encrypt_dict[char]
    else:
        ciphertext += char

print("\nEncrypted Message:", ciphertext)

# Decryption

decrypted = ""

for char in ciphertext:

    if char in decrypt_dict:
        decrypted += decrypt_dict[char]
    else:
        decrypted += char

print("Decrypted Message:", decrypted)
```

Sample Output

```
Enter Message: HELLO

Encrypted Message: ITSSG
Decrypted Message: HELLO
```

```python
# ================================================
# SUBSTITUTION CIPHER IMPLEMENTATION
# ================================================

alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

substitution = "QWERTYUIOPASDFGHJKLZXCVBNM"

encrypt_dict = {}

decrypt_dict = {}

# Create Encryption Dictionary

for i in range(len(alphabet)):
    encrypt_dict[alphabet[i]] = substitution[i]

# Create Decryption Dictionary

for i in range(len(alphabet)):
    decrypt_dict[substitution[i]] = alphabet[i]

# Input

plaintext = input("Enter Message: ").upper()

# Encryption
```



```python
plaintext = input("Enter Message: ").upper()

# Encryption

ciphertext = ""

for char in plaintext:

    if char in encrypt_dict:
        ciphertext += encrypt_dict[char]
    else:
        ciphertext += char

print("\nEncrypted Message:", ciphertext)

# Decryption

decrypted = ""

for char in ciphertext:

    if char in decrypt_dict:
        decrypted += decrypt_dict[char]
    else:
        decrypted += char

print("Decrypted Message:", decrypted)
```


Comparison of Classical Encryption Techniques

Feature	Caesar Cipher	Substitution Cipher
Key Type	Numeric Shift	Character Mapping
Security	Low	Moderate
Complexity	Simple	Moderate
Encryption Speed	Fast	Fast
Vulnerability	Brute Force	Frequency Analysis

Advantages

Caesar Cipher

- Easy to implement.
- Simple encryption method.
- Useful for learning cryptography.

Substitution Cipher

- More secure than Caesar Cipher.
- Larger key space.
- Better character substitution mechanism.

Limitations

Caesar Cipher

- Easily broken by brute-force attack.
- Limited security.

Substitution Cipher

- Vulnerable to frequency analysis.
- Not suitable for modern security requirements.

Applications

- Learning cryptography concepts.
- Educational demonstrations.
- Historical encryption studies.
- Understanding modern cryptographic foundations.

Result

The Caesar Cipher and Substitution Cipher algorithms were successfully implemented. Encryption and decryption operations were performed, demonstrating the working principles of classical cryptographic techniques.